

1 Database theory.

1.1 Primary key/Foreign keys.

```
CREATE TABLE Reservation (  
    client int ,  
    room int ,  
    arrival datetime ,  
    PRIMARY KEY ( room , arrival ) ,  
    FOREIGN KEY ( client ) REFERENCES Client ( id )  
);
```

1.2 Relational algebra.

Operations:

- $\langle j : d \rangle$ is “tuple creation”
- R , for $R \in \mathcal{R}$
- $\sigma_{j=d}$ is *selection*
- π_X is *projection*
- ρ_f is *renaming*
- $- \bowtie -$ is *natural join*
- $- \cup -$ is *union*
- $- \setminus -$ is *difference*

Other operations:

- $- \div -$ is *division*: writing $\Delta := \text{Att}_R \setminus \text{Att}_S$,

$$R \div S := \{u \in \pi_\Delta(R) \mid \forall s \in S, su \in R\}$$

- $- \ltimes -$ is *semi-join*, $R \ltimes S := \pi_{\text{Att}_R}(R \bowtie S)$
- θ -join is a join based on *inequalities*, i.e. $\sigma_{\text{condition}}(R \bowtie S)$

1.3 First-order logic.

- **Semantics.** Given an assignment $\lambda : \text{Vars} \rightarrow \text{Domain}$, we write $I, \lambda \models \phi$.
- $\llbracket \phi \rrbracket^I := \{\lambda : \text{FreeVars}(\phi) \rightarrow \text{Domain} \mid I, \lambda \models \phi\}$
- Query *evaluation* is **PSPACE**-complete (is $\llbracket \phi \rrbracket^I$ non-empty?)
- Query *containment* is **undecidable** (is $\llbracket \phi \rrbracket^I$ a subset of $\llbracket \psi \rrbracket^I$ for all I ?)
- Query *equivalence* is **undecidable** (is $\llbracket \phi \rrbracket^I$ equal to $\llbracket \psi \rrbracket^I$ for all I ?)
- *Satisfiability* (with finite model) is **undecidable** (Trakhenbrot’s theorem).
- Domain independance is **undecidable**

1.4 Codd's theorem.

Domain independent first-order logic is equivalent to relational algebra.

- From SPJRUD to FO, use $\exists$ as a mask.
- From FO to SPJRUD, we need to use the *active domain* Adom_x :
 - $\neg\phi \rightsquigarrow \prod_{x \in \text{FreeVars}(\phi)} \text{Adom}_x \setminus q_\phi$
 - $x = y \rightsquigarrow \sigma_{x=y} \text{Adom}_x \bowtie \text{Adom}_y$
 where $\text{Adom}_x := \left(\bigcup_{a \in \text{adom}(\phi)} \langle x : a \rangle \right) \cup \left(\bigcup_{R \in \mathcal{R}} \bigcup_{u \in \text{Att}_R} \pi_x \rho_{u \mapsto x} R \right)$.

1.5 Conjunctive queries.

Only use $\exists$ and $\wedge$ (no $\vee$, $\neg$ nor $\forall$). This corresponds to SPJR (no union and no difference, obviously).

We write these formulas with prefix $\exists$ s (leaving them implicit).

All conjunctive queries are *domain independent* (if range-restricted) and *satisfiable* (with the *canonical database* $I[q]$).

Range-restricted. We allow “ $x = y$ ” (with x, y variables) in the query as long as we have x or y in some $R(\dots)$.

General form is $q(\underbrace{y_1, \dots, y_n}_{\text{head}}) := \underbrace{R_1(\vec{u}_1) \wedge \dots \wedge R_m(\vec{u}_m)}_{\text{body}}$.

A *boolean* conjunctive query q has no head variable (and thus $\llbracket q \rrbracket^I$ is either empty or singleton).

Homomorphism theorem. The three are equivalent:

- query q is contained in query q' ;
- there exists a homomorphism $h : q' \rightarrow q$;
 - $h : \text{Vars}(q') \rightarrow \text{Vars}(q) \cup \text{Constants}(q)$
 - for all $R(\vec{u})$ in q' there exists a $R(h(\vec{u}))$ in q
 - $h(\text{HeadVars}(q')) = \text{HeadVars}(q)$.
- $\text{HeadVars}(q)$ satisfies q' in the canonical database $I[q]$.

Query evaluation and containment are **NP**-complete (by homomorphism theorem).

1.5.1 Acyclic conjunctive queries.

With Yannakakis's algorithm, query evaluation is in **P**:

- Construct a *join-tree*
- Evaluate the query following the join-tree (using semijoins).

Constructing the join-tree (*Graham-Yu-Ozsoyoglu*). Consider the hypergraph where hyperedges are exactly relations and vertices are $\text{Vars}(q)$. The idea is to

remove *ears*. An *ear* e is an hyperedge which is either isolated or has a face (a *face* f is an hyperedge such that vertices $e \setminus f$ only occurs in e , like a face).

1.6 Dependencies.

1.6.1 Functional dependencies.

A *functional dependency* is an expression $R : X \rightarrow Y$ where $X, Y \subseteq \text{Att}_R$, meaning X *uniquely depends* on Y .

Semantics. We have $I \models R : X \rightarrow Y$ iff, for all $r, r' \in R^I$, if $r|_{\text{Att}_X} = r'|_{\text{Att}_X}$ then $r|_{\text{Att}_Y} = r'|_{\text{Att}_Y}$.

A *superkey* $K \subseteq \text{Att}_R$ for R is a functional dependency $R : K \rightarrow \text{Att}_R$. A *candidate key* is a superkey such that none of its proper subsets is a superkey.

Armstrong's axioms.

$$\begin{array}{c} \frac{Y \subseteq X}{X \rightarrow Y} \text{reflexivity} \quad \frac{X \rightarrow Y}{XZ \rightarrow YZ} \text{augmentation} \\ \frac{X \rightarrow Y \quad Y \rightarrow Z}{X \rightarrow Z} \text{transitivity} \end{array}$$

extensions:

$$\frac{X \rightarrow Y \quad X \rightarrow Z}{X \rightarrow YZ} \text{union} \quad \frac{X \rightarrow YZ}{X \rightarrow Y} \text{decomposition}$$

The *closure* of X under Σ (set of functional dependencies) is

$$\mathcal{C}\ell_{\Sigma}(X) := \{A \text{ attribute} \mid \Sigma \models X \rightarrow A\}.$$

It can be computed in linear-time.

Boyce-Codd normal form. For all $X, Y \subseteq \text{Att}_R$, if $\Sigma \models R : X \rightarrow Y$ then $Y \subseteq X$ or X is superkey for R .

1.6.2 Join dependencies.

A *join dependency* is an expression $R : \bowtie[X_1, \dots, X_n]$.

Semantics. $I \models R : \bowtie[X_1, \dots, X_n]$ iff $R^I = \pi_{X_1}(R^I) \bowtie \dots \bowtie \pi_{X_n}(R^I)$.

1.6.3 Multi-Valued dependencies.

A *multi-valued dependency* is an expression $R : X \twoheadrightarrow Y$. The meaning of a multi-valued dependency $R : X \twoheadrightarrow Y$ is a join-dependency $R : \bowtie[XY, X\bar{Y}]$ where $\bar{Y} := \text{Att}_R \setminus Y$.

Important. $I \models R : X \rightarrow Y$ implies $I \models R : X \twoheadrightarrow Y$.

Using Armstrong's rules, with the following four for mvd

$$\begin{array}{c}
\frac{Y \subseteq X}{X \twoheadrightarrow Y} \text{reflexivity} \qquad \frac{X \twoheadrightarrow Y}{XZ \twoheadrightarrow YZ} \text{augmentation} \\
\frac{X \twoheadrightarrow Y}{X \twoheadrightarrow Y} \text{"symmetry"} \qquad \frac{X \twoheadrightarrow Y \quad Y \twoheadrightarrow Z}{X \twoheadrightarrow Z \setminus Y} \text{transitivity}
\end{array}$$

and the following two for mvd $\leftrightarrow$ fd:

$$\frac{X \rightarrow Y}{X \twoheadrightarrow Y} \qquad \frac{X \twoheadrightarrow Y \quad XY \rightarrow Z}{X \rightarrow Z \setminus Y}$$

we obtain a **sound and complete** axiomatization.

1.6.4 Chase algorithm.

- Start with a table containing two tuples with all different values;
- when considering hypothesis $A \rightarrow BC$, we will *add equalities*: if two tuples have the same X -values, then make their Z -values equal;
- when considering hypothesis $X \twoheadrightarrow Z$, we will *add tuples*: if two tuples have the same X -values, then add two copies of these two tuples and exchange their Z -values.

1.7 Recursive queries.

```

WITH RECURSIVE T( x1 , ... , xn ) AS (
  SELECT -- < base case query >
UNION
  SELECT -- < inductive case query using T >
)
SELECT * FROM T ;

```

This uses algorithms to compute a fixed-point (we should consider a *monotone* map $F : 2^{\mathcal{U}} \rightarrow 2^{\mathcal{U}}$). In that case, the “trivial” fixed-point algorithm always terminates (for finite $\mathcal{U}$) and returns $\text{lfp}(F)$.

- *Semipositive Datalog*: monotone so $\checkmark$
- *Datalog $^\neg$* : non-monotone but we can use *inflationary semantics* so $\checkmark$
- *Datalog $^\neg$* : partial fixed-points.

Equivalence between Datalog $^\neg$ and infl. while programs. These programs have the form

“while something changed, do ... $\cup$ = ... (a few times)”.

Main results.

- $\text{FO}[\text{LFP}, <] = \text{FO}[\text{LFP}, <] = \mathbf{P}$.
- $\text{FO}[\text{PFP}, <] = \mathbf{PSPACE}$.

1.8 Size of joins.

AGM bound. Let q a full CQ. For all fractional edge covers u of $G(q)$, we have:

$$\# \llbracket q \rrbracket^I \leq \prod_{R(\vec{x}) \in \text{body}(q)} (\# R^I)^{u(R(\vec{x}))}.$$

A *fractional edge cover* of an hypergraph (E, V) is $u : E \rightarrow \mathbb{R}^+$ such that, for all $x \in V$,

$$\sum_{e \ni x} u(e) \geq 1.$$

1.9 Provenance.

Fix K a *semiring*. A K -relation is a map $R : \text{Domain}^{\text{Att}_R} \rightarrow K$ whose support is finite (only finite non-zero values).

Provenance is inductively defined by

- $\text{Prov}(\emptyset) := t \mapsto 0$.
- $\text{Prov}(q \cup q') := t \mapsto \text{Prov}(q)(t) + \text{Prov}(q')(t)$.
- $\text{Prov}(\sigma_P q) := t \mapsto \begin{cases} \text{Prov}(q)(t) & \text{if predicate } P(t) \text{ holds} \\ 0 & \text{otherwise} \end{cases}$.
- $\text{Prov}(\pi_A q) := t \mapsto \sum_{t' \text{ st } t'_{|A} = t_{|A}} \text{Prov}(q)(t')$.
- $\text{Prov}(q \bowtie q') := t \mapsto \text{Prov}(q)(t_{| \text{Att}_q}) \times \text{Prov}(q')(t_{| \text{Att}_{q'}})$
- $\text{Prov}(q \setminus q') := t \mapsto \text{Prov}(q)(t) - \text{Prov}(q')(t)$

2 Data Mining.

Vocabulary. data set, data object, attribute/dimension/feature/variable (nominal, binary, ordinal, quantity, interval, ratio, discrete, continuous).

- Measuring the Central Tendency: mean, median, mode (most frequent value)
- Measures Data Distribution:
 - Variance/Standard Deviation,
 - Covariance ($\sigma_{1,2} = \mathbb{E}[X_1 X_2] - \mu_1 \mu_2$, **warning** $\sigma_{1,1} = \sigma_1^2$) $\rightsquigarrow$ *covariance matrix*,
 - $\chi^2 = \sum_i \sum_j (o_{i,j} - e_{i,j})^2 / e_{i,j}$ where $o_{i,j}$ is the *observed* $\#(A_i, B_j)$ and $e_{i,j}$ is the *expected* $\#A_i \times \#B_j / n$,
 - Correlation $\rho_{i,j} = \sigma_{i,j} / \sigma_i \sigma_j$
 - Dissimilarity matrix $(\text{dist}(\vec{x}_i, \vec{x}_j))_{i,j \in [1,n]}$, requires a meaningful distance $\text{dist}(\cdot, \cdot)$ (e.g. the Minkowski distance for numeric data)
 - z-score: $z = (x - \mu) / \sigma$.

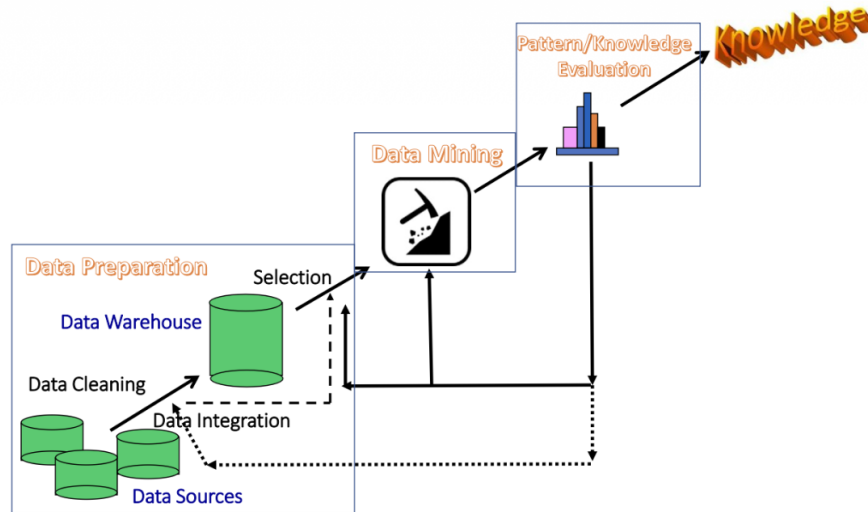


Figure 1: Everything in one figure

- Graphic Displays of Basic Statistical Descriptions: Boxplot, Histogram, Quantile plot, Quantile-quantile (q-q) plot, Scatter plot.

A **document** is a matrix labels $\times$ frequencies.

KL (Kullback-Leibler) **Divergence**. Comparing Two Probability Distributions.

$$D_{\text{KL}}(p||q) = \sum_{x \in \mathcal{X}} p(x) \ln \left(\frac{p(x)}{q(x)} \right).$$

“How many bits should I add to $q(\cdot)$ to obtain $p(\cdot)$?”

2.1 Data preprocessing.

- Data cleaning (*e.g.* removing outliers, dealing with missing data)
- Data integration (*e.g.* data cubes)
- Data reduction (*e.g.* dimensionality reduction, data compression, **sampling**)
- Data transformation and data discretization (*e.g.* normalization).

2.1.1 Data cleaning.

- How to Handle Missing Data? Use the most probable value.
- How to Handle Noisy Data? Binning, Regression, Clustering, Semi-supervised

2.1.2 Data transformation.

- Normalization: min-max normalization, z -score normalization, decimal scaling $x' = x/10^j$.
- Discretization: binning, histogram, clustering, decision-tree (supervised), correlation.

2.2 Data Warehouse.

- **OLTP.** “Analysis while the data is collecting”
- **OLAP.** “Analysis after the data is collected”

We should use a data warehouse for **OLAP**, not **OLTP**.

“Usual” operations. Sort, summarize, consolidate, compute views, check integrity, and build indices and partitions.

Refresh: propagate the updates from the data sources to the warehouse

A *data lake* is a centralized repository storing all structured and unstructured data at any scale in an organization. Everything in one place, stored “as is”, no analysis.

2.3 Data cubes.

Data that can be viewed in multiple dimensions:

- a set of tables for *dimensions* (one “side” of a cube)
- a table of *facts* ($\sim$ raw data)

An n -dimensional base cube is called a *cuboid*. All cuboids form a *lattice*.

A few schema: *star* schema, *snowflake* schema, *fact constellation*.

Data Cube Measures.

- *Distributive*: apply to groups and compare with global.
- *Algebraic*: computed by an algebraic function.
- *Holistic*: **no idea**

Operations.

- *Roll up (drill-up)*: summarize data by climbing up hierarchy or by dimension reduction
- *Drill down (roll down)*: reverse of roll-up, from higher level summary to lower level summary or detailed data, or introducing new dimensions

Iceberg cube. Compute only the cells whose measure satisfies the iceberg condition (some condition to filter)

2.4 Pattern mining.

Pattern discovery $\rightsquigarrow$ classification, clustering.

A k -itemset is an itemset with k items. Absolute Support: number of occurrences.

Relative Support: probability.

Downward closure (also called “*apriori*”): any subset of a frequent itemset must be frequent

How to compute frequent k -itemsets:

- Scan DB once to get frequent 1-itemset
- While no frequent or candidate set can be generated do
 - Generate length- $(k + 1)$ candidate itemsets from length- k frequent itemsets
 - Test the candidates against DB to find frequent $(k + 1)$ -itemsets
 - Increase k

FP-trees (FP means frequent patterns). Allow for divide and conquer.

- Supervised learning (classification, numeric prediction): we give labels
- Unsupervised learning (clustering): labels are unknown

2.5 Supervised Classification.

Bayes’ theorem.

$$\underbrace{\Pr[H \mid X]}_{\text{posteriori}} \propto \underbrace{\Pr[X \mid H]}_{\text{likelihood}} \times \underbrace{\Pr[H]}_{\text{prior}}$$

Classification is to derive the maximum posteriori. We use “likelihood tables.”

Decision trees induced by the data (ID3): at each node we try to maximize the *information gain*. For *continuous values*, use **best split point**. They are easy to explain, implement, efficient but lack stability as they tend to overfit the data.

Lazy/eager learners.

- *lazy*: no preprocessing, full algorithm at runtime (*e.g.* k -NN);
- *eager*: preprocessing and minimal algorithm at runtime.

Confusion matrix. True/false positive/negative.

2.6 Clustering.

- Essential Measures of Cluster:
 - **Centroid.** $\text{centroid}(C) = \text{avg } C$
 - **Radius.** $\text{radius}(C) = \sqrt{\text{avg}_{x \in C} \text{dist}(x, \text{avg } C)^2}$
 - **Diameter.** $\text{diameter}(C) = \sqrt{\text{avg}_{x \neq x' \in C} \text{dist}(x, x')^2}$

2.6.1 *k*-means.

Complexity in $O(tkn)$ for t iterations and n elements, usually $k, t \ll n$.

Cons:

- Usually stuck on a local optimum,
- k needs to be specified correctly,
- sensitive to noise and outliers,
- only for *continuous* attributes in convex spaces.

Improvement: *k*-means++. Initialize one centroid at a time, choosing the furthest one from all (already initialized) others at each step.

Improvement: *k*-medoids. Instead of mean use medoid in the iteration loop. Pick a random point non-repr, and compute the cost of swapping representative; swap if the cost is negative. Complexity in $O(k(n-k)^2)$.

Random empirical value. Pick k around $\sqrt{n/2}$.

2.6.2 Agglomerative Clustering.

- Single link (nearest neighbor) $\rightsquigarrow \text{sim}(C, C') = \min_{(x, x') \in C \times C'} \text{sim}(x, x')$
- Complete link (diameter) $\rightsquigarrow \text{sim}(C, C') = \max_{(x, x') \in C \times C'} \text{sim}(x, x')$
- Average link (group average) $\rightsquigarrow \text{sim}(C, C') = \text{avg}_{(x, x') \in C \times C'} \text{sim}(x, x')$
- Centroid link (centroid similarity) $\rightsquigarrow \text{sim}(C, C') = \text{sim}(\text{avg } C, \text{avg } C')$.

2.6.3 BIRCH: A Multi-Phase Hierarchical Clustering Method.

- **Phase 1.** Build a CF (clustering feature) Tree.
- **Phase 2.** Arbitrary algorithm to cluster the leaf nodes of the tree.

Building the CF tree is based on

For each point in the input,

- Find its closest leaf entry
- Add point to leaf entry and update CF
- If entry diameter $> \text{max_diameter}$ then split leaf, and possibly parents

Lead to unnatural clusters as we fix the *size* of each cluster not the number of clusters.

Random empirical value. Pick $\sqrt{2n}$ for the maximum size of each cluster.

2.6.4 DBSCAN: Density-Reachable and Density-Connected.

A cluster is defined as a maximal set of density-connected points. Two parameters:

- radius ε of neighborhood
- minimum number of points on the neighborhood

- $\text{neighborhood}(x) := \{y \in \mathcal{D} \mid \text{dist}(x, y) \leq \varepsilon\}$.

Complexity in $O(n^2)$ (or $O(n \log n)$ if spatial index).

2.6.5 OPTICS: Ordering Points To Identify Clustering Structure.

Extension of *DBSCAN*.

Idea. Higher density points should be processed first; find high-density clusters first.

Same complexity.

2.6.6 Grid-based Clustering Methods.

- **STING: STatistical INformation Grid approach.** Different levels of resolution, like a CF-tree.
- **CLIQUE.** Find dense regions (clusters) in each subspace; use the apriori principle: a 2D region can't be dense if its projection is not.